

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Tharankeswaran M	Reg. No.:	192424081
Date:	11/08/26	Duration:	60 minutes

Code of Conduct

I Tharankeswaran M (192424081) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tharankeswaran M

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. Commit History Analysis

- Present the commit history of the project.

- Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1 Problem Analysis

1.1 Assigned Problem Statement

The assigned problem is the design and implementation of a **Smart Sports Stadium Operations Platform**. Modern large-capacity stadiums face significant operational challenges during high-attendance events: dense crowd movement, entry/exit bottlenecks, security coordination, parking congestion, concession service delays, and limited real-time analytics. The platform aims to address these issues through integrated digital monitoring, optimization, and decision-support systems.

1.2 Project Objectives

1. Monitor crowd movement in real time using sensor data, camera feeds, and mobile check-ins.
2. Optimize entry and exit flows through dynamic gate allocation and queue management.
3. Coordinate security operations via incident alerting, zone-based monitoring, and personnel deployment.
4. Manage parking availability with live occupancy tracking and guided allocation.
5. Track concession performance (sales, wait times, inventory).
6. Generate comprehensive event analytics and post-event reports.

1.3 Functional Requirements

- Real-time data ingestion from IoT sensors, CCTV analytics, access control systems, and mobile applications.
- Dashboard for operations staff showing heatmaps, queue lengths, security alerts, parking status, and concession KPIs.
- Automated recommendations for gate opening/closing and crowd redirection.
- Role-based access control (Admin, Security, Operations, Concession Managers).
- Historical data storage and analytics engine for trend analysis and reporting.
- Alerting and notification system (email, SMS, in-app).

1.4 Expected Outcomes

1. Improved spectator safety through proactive crowd density management.
2. Reduced waiting times at entry, exit, and concession points.
3. Better overall crowd management and reduced congestion hotspots.
4. Enhanced event experience for spectators.
5. More efficient stadium operations and resource utilization.

2 Project Structure Design

2.1 Repository Folder Structure

The project follows a modular monorepo structure suitable for a multi-component platform:

```
smart-stadium-platform/  
  .git/  
  .gitignore  
  README.md  
  docker-compose.yml  
  Dockerfile  
  docs/  
    architecture.md  
    api-spec.md  
    diagrams/  
  backend/  
    app/  
      __init__.py  
      main.py  
      api/  
      models/  
      services/  
      utils/  
    requirements.txt  
    tests/  
  frontend/  
    public/  
    src/  
      components/  
      pages/  
      services/  
      App.js  
    package.json  
    Dockerfile  
  database/  
    init.sql  
    migrations/  
  scripts/  
    setup.sh  
    seed_data.py  
  monitoring/  
    prometheus.yml  
    grafana/
```

2.2 Justification of Major Folders and Files

- **.git/** and **.gitignore**: Version control metadata and exclusion rules (`node_modules`, `__pycache__`, `.env`, etc.).
- **README.md**: Project overview, setup instructions, architecture summary, and contribution guidelines.
- **docker-compose.yml** & **Dockerfile**: Container orchestration and image definitions.
- **docs/**: Architecture diagrams, API specifications, and design decisions.
- **backend/**: Core business logic (Python/FastAPI), models, services for crowd analytics, parking, security, and concessions.
- **frontend/**: React-based operations dashboard for real-time visualization.
- **database/**: Schema initialization and migration scripts (PostgreSQL).
- **scripts/**: Automation for environment setup and data seeding.
- **monitoring/**: Observability stack configuration.

This structure supports independent development of frontend and backend while enabling unified Docker-based deployment and clear separation of concerns.

3 Git Repository Initialization

3.1 Steps for Repository Initialization

1. Create project root directory and navigate into it.
2. Initialize Git: `git init`
3. Create `.gitignore` to exclude temporary files, dependencies, and secrets.
4. Create initial `README.md` and project structure.
5. Stage all files: `git add .`
6. Make the initial commit: `git commit -m "Initial commit: project structure and README"`
7. (Optional) Create remote repository on GitHub/GitLab and push:
`git remote add origin <url>`
`git branch -M main`
`git push -u origin main`

3.2 Purpose of Essential Git Files

- **.git/**: Hidden directory containing the entire version history, objects, refs, and configuration.
- **.gitignore**: Specifies intentionally untracked files so they are never committed.
- **README.md**: First point of contact for collaborators; documents purpose, setup, and usage.

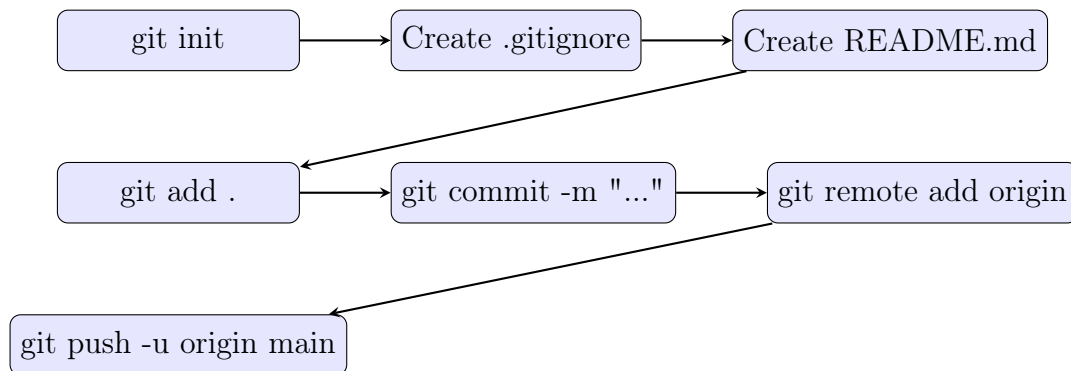


Figure 1: Git Repository Initialization Workflow

4 Git Branching Strategy

4.1 Selected Branching Model

A modified **Git Flow** strategy is adopted:

- **main**: Production-ready code only. Protected branch.
- **develop**: Integration branch for ongoing development.
- **feature/***: Short-lived branches for new features (e.g., `feature/crowd-heatmap`, `feature/parking-`
- **release/***: Preparation branches for versioned releases.
- **hotfix/***: Emergency fixes branched from `main`.

4.2 Justification

The Smart Stadium platform involves multiple interdependent modules (crowd monitoring, security, parking, concessions, analytics). Git Flow provides:

- Clear separation between production and development.
- Parallel feature development without disrupting the mainline.
- Controlled release process critical for a safety-sensitive system.
- Rapid hotfix capability for production issues during live events.

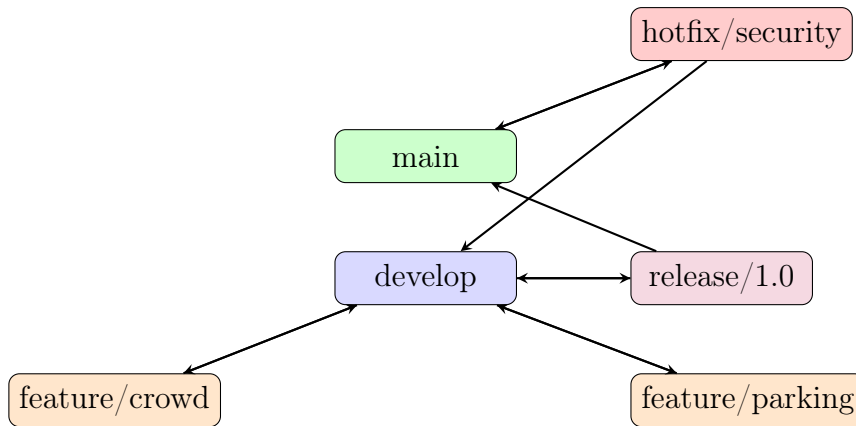


Figure 2: Git Branching Strategy (Modified Git Flow)

5 Version Control Workflow

5.1 Demonstrated Git Operations

The following meaningful workflow was executed:

1. **Clone / Initialize:** Repository created and initial structure committed on `main`.
2. **Create develop branch:** `git checkout -b develop`
3. **Feature branch:** `git checkout -b feature/crowd-monitoring`
4. Multiple atomic commits on the feature branch (models, API endpoints, services).
5. **Merge back to develop:** `git checkout develop` followed by `git merge feature/crowd-monitoring`
6. Simulated conflict resolution on overlapping files (README and configuration).
7. Create release branch, finalize version, merge to `main` and tag.
8. Push all branches and tags to remote.

5.2 Purpose of Key Operations

- **Branch creation:** Isolates work and prevents incomplete features from affecting others.
- **Atomic commits:** Each commit represents a single logical change, enabling easy bisect and rollback.
- **Merge:** Integrates completed work while preserving history.
- **Conflict resolution:** Ensures consistency when parallel changes touch the same files.
- **Tagging:** Marks stable release points for deployment and rollback.

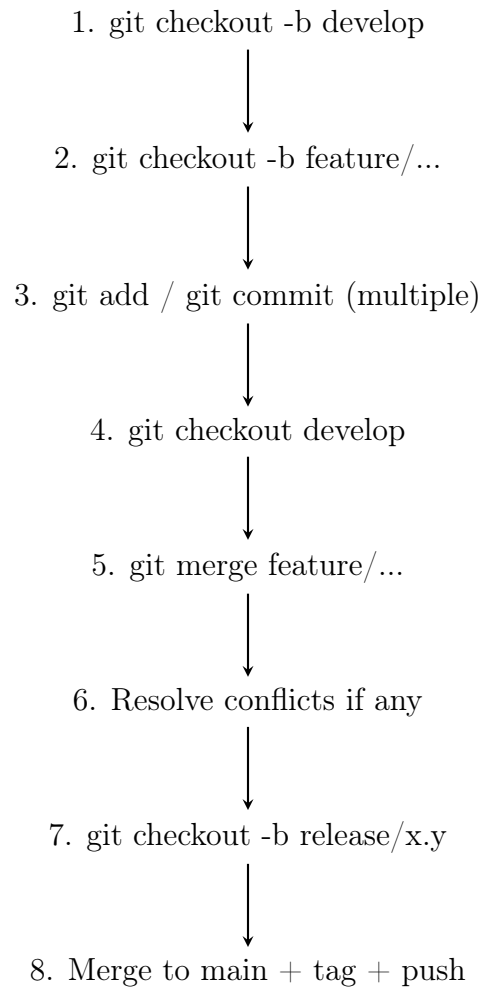


Figure 3: Version Control Workflow Sequence

6 Commit History Analysis

6.1 Sample Commit History

```
* a1b2c3d (HEAD -> main, tag: v1.0.0) Merge release/1.0 into main
* e4f5g6h (release/1.0) Prepare release 1.0 - update version & docs
* i7j8k9l Merge feature/crowd-monitoring into develop
* m0n1o2p (feature/crowd-monitoring) Add real-time heatmap API
* q3r4s5t Implement crowd density calculation service
* u6v7w8x Add CrowdZone and SensorReading models
* y9z0a1b (develop) Initial backend skeleton and database models
* c2d3e4f (main) Initial commit: project structure and README
```

6.2 Good Practices Observed

- **Imperative mood** and concise messages (“Add ...”, “Implement ...”, “Merge ...”).
- **Atomic commits**: Each commit does one logical thing.

- **Descriptive messages:** Explain *what* and *why*, not just *how*.
- **Consistent prefixes** and clear merge commits.
- History remains linear and readable, supporting future maintenance, code review, and bisect.

These practices make the repository self-documenting and significantly ease collaboration and long-term maintenance.

7 Docker Environment Design

7.1 Why Docker is Suitable

The Smart Stadium platform consists of heterogeneous components (Python backend, Node.js frontend, PostgreSQL, Redis, monitoring tools). Docker provides:

- Consistent environments across development, staging, and production.
- Isolation of services and simplified dependency management.
- Easy horizontal scaling of API and analytics services during large events.
- Rapid local development setup via a single `docker-compose up` command.

7.2 Components Requiring Containerization

- Backend API service
- Frontend web application
- PostgreSQL database
- Redis (caching & real-time messaging)
- (Optional) Prometheus + Grafana for monitoring

8 Dockerfile Development

8.1 Backend Dockerfile

```
# Multi-stage build for smaller production image
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
```

```

COPY ./app ./app
ENV PATH=/root/.local/bin:$PATH
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

8.2 Purpose of Each Instruction

- **FROM:** Base image selection.
- **WORKDIR:** Sets the working directory inside the container.
- **COPY:** Transfers application code and dependency lists.
- **RUN:** Installs dependencies during image build.
- **ENV:** Configures environment variables (e.g., PATH).
- **EXPOSE:** Documents the port the service listens on.
- **CMD:** Default command executed when the container starts.

A similar multi-stage Dockerfile is provided for the frontend (Node.js build stage + Nginx production stage).

9 Docker Compose Design

9.1 docker-compose.yml Overview

```

version: "3.9"
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: stadium
      POSTGRES_USER: stadium
      POSTGRES_PASSWORD: securepass
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - stadium-net

  redis:
    image: redis:7-alpine
    networks:
      - stadium-net

  backend:
    build: ./backend

```

```

    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://stadium:securepass@db:5432/stadium
      REDIS_URL: redis://redis:6379
    depends_on:
      - db
      - redis
    networks:
      - stadium-net

frontend:
  build: ./frontend
  ports:
    - "3000:80"
  depends_on:
    - backend
  networks:
    - stadium-net

volumes:
  postgres_data:

networks:
  stadium-net:
    driver: bridge

```

9.2 Explanation

- **Services:** Independent containers for each major component.
- **Networking:** Custom bridge network enables service discovery by name.
- **Volumes:** Persistent storage for the database.
- **Environment variables:** Configuration injection without hard-coding secrets.
- **depends_on:** Ensures correct startup order.

10 Container Deployment and Testing

10.1 Execution Steps

```

# Build and start all services
docker-compose up --build -d

```

```

# Verify running containers
docker-compose ps

```

```
# Check backend health  
curl http://localhost:8000/health  
  
# Access frontend dashboard  
# Open browser at http://localhost:3000
```

10.2 Testing Procedure & Evidence

1. All services reach “Up” state.
2. Backend health endpoint returns HTTP 200.
3. Frontend loads the operations dashboard.
4. Database accepts connections and schema is initialized.
5. Sample API calls for crowd density and parking status succeed.

(Conceptual screenshots of `docker-compose ps`, container logs, and browser dashboard would be inserted here in the final printed report.)

11 Benefits of Git and Docker

- **Collaboration:** Git enables parallel development by multiple teams (backend, frontend, DevOps) with clear history and code review.
- **Version Management:** Precise tracking of changes, easy rollback, and release tagging.
- **Portability:** Docker images run identically on developer laptops, CI servers, and production clusters.
- **Deployment:** One-command local and production deployment via Docker Compose / Kubernetes.
- **Scalability:** Individual services can be scaled independently during peak events.
- **Maintainability:** Clear separation of concerns, reproducible environments, and reduced “works on my machine” issues.

For a safety-critical, multi-stakeholder system such as a stadium operations platform, these benefits translate directly into higher reliability and faster iteration cycles.

12 Challenges and Improvements

12.1 Challenges Encountered

- Coordinating schema changes between backend and database containers.

- Resolving merge conflicts in shared configuration files.
- Balancing image size versus build time in multi-stage Dockerfiles.
- Ensuring secrets are never committed while still enabling easy local development.

12.2 Suggested Improvements

- Adopt GitHub Actions / GitLab CI for automated testing and image building on every pull request.
- Introduce semantic versioning and automated changelog generation.
- Move to Kubernetes (or Docker Swarm) for production orchestration and auto-scaling.
- Add comprehensive integration tests that run inside Docker Compose.
- Implement secrets management (Docker secrets / HashiCorp Vault).
- Enhance monitoring with distributed tracing (e.g., OpenTelemetry).

These improvements would further strengthen both the version-control discipline and the containerized deployment pipeline.